CS527 - Computer Systems Lab: OpenMP Arrays & Matrices

Name: Fardeen Khan Nabi Khan

Entry Number: 2026CSM1013

GitHub Repository Link: https://github.com/FARDeen9307/Computer-System-Parallel-Programming

1. Introduction & Overview

This report presents the implementation details, execution outputs, and performance benchmarking results for Assignment 2 of the Computer Systems Lab, focusing on OpenMP Parallel Programming for Arrays and Matrices. The implementation covers serial vs. parallel element matching, parallel sorting via odd-even transposition, thread-safe histogram calculation using atomic updates, parallel 2D matrix transposition, and parallel 2D convolution.

Timing Rule Compliance

In strict adherence to high-performance computing timing guidelines:

- **Timer:** The wall-clock execution timer begins immediately before invoking the target parallel or serial function (omp_get_wtime()) and stops immediately after its completion.
- **Excluded work:** Dynamic memory allocation (new/delete[]), data initialization, verification checks, and terminal printing are completely excluded from the measured execution time.
- **Build:** All reported runtime values represent stable, verified executions compiled with g++ -fopenmp -O2 -Wall.

2. Directory Structure & Execution Setup

2.1 Project Layout

Computer-System-Parallel-Programming/

```
│
├── lab 2 tasks/
│   ├── task1.cpp  # Array Search (Serial vs. Parallel Index Matching)
│   ├── task2.cpp  # Array Sorting (Odd-Even Transposition Sort)
│   ├── task3.cpp  # Histogram Calculation (Values 0-9 with Atomic Updates)
│   ├── task4.cpp  # Matrix Transpose (5000x5000 and 10000x10000 Grid)
│   └── task5.cpp  # 2D Convolution (3x3 Kernel on Image Matrix)
│
│
└── README.md
```

2.2 Execution Setup & Commands

Thread counts were managed dynamically prior to running the respective compiled binaries via PowerShell:

```
# Compilation Example
g++ -fopenmp -O2 task1.cpp -o task1
```

```
# Setting Thread Count & Execution
$env:OMP_NUM_THREADS = "8"
./task1
```

Task 1: Array Search

1. Objective & Problem Statement

The goal of this task is to implement and compare serial and parallel search algorithms in a large array a[N] (tested across sizes N=10^7 and N=10^8) to locate a specific target value and determine its exact index using thread-safe synchronization (critical or atomic) while measuring scaling performance across multiple thread counts (P=1,2,4,8).

2. Implementation Details

- **Compilation & Execution:** Compiled using g++ -fopenmp -O2 -Wall and execution time measured precisely via wall-clock timer omp_get_wtime(). Thread counts were managed dynamically using $env:OMP_NUM_THREADS.
- **Serial Version:** A sequential linear scan traversing array indices from 0 to N-1 to locate the target value and return its index.
- **Parallel Version:** Parallelized using #pragma omp parallel for combined with thread-safe control structures (#pragma omp critical) to record the matched index without data races.
- **Verification:** Validates that the index returned by the parallel search identically matches the serial search baseline across all test runs, outputting SUCCESS.

3. Timing and Speedup Table

(Measured across array sizes N=10^7, 10^8 and thread counts P=1,2,4,8)

Size (N)	Threads	Tser (s)	Tpar (s)	Speedup	Status
10^7	1	0.0030	0.0070	0.43	SUCCESS
10^7	2	0.0040	0.0050	0.80	SUCCESS
10^7	4	0.0020	0.0020	1.00	SUCCESS
10^7	8	0.0040	0.0020	2.00	SUCCESS
10^8	1	0.0200	0.0470	0.43	SUCCESS
10^8	2	0.0230	0.0190	1.21	SUCCESS
10^8	4	0.0180	0.0110	1.64	SUCCESS
10^8	8	0.0220	0.0290	0.76	SUCCESS

4. Observations

- **Why is parallel execution sometimes slower than serial execution when using 1 thread (P=1)?**
  - **Answer:** When running with a single thread, the parallel version incurs additional runtime overhead due to OpenMP environment setup, work-sharing construct management, and thread scheduling directives. Because the actual linear search computation on contiguous memory is extremely fast (taking only milliseconds), this structural overhead accounts for a noticeable fraction of the total execution time, making the pure serial loop slightly faster.
- **How does performance scale with thread count, and why does speedup fluctuate or drop at higher core counts (e.g., at P=8 for N=10^8)?**
  - **Answer:** Array searching is fundamentally a memory-bandwidth bound operation. As moderate thread counts (P=2,4) are utilized, parallelizing the search helps distribute the load and hide cache/memory latency, yielding speedups up to 1.64x. However, at higher core counts (such as P=8 on N=10^8), multiple threads simultaneously saturate the system memory bus bandwidth as they fetch independent chunks of the array. This creates memory bus contention and cache thrashing, leading to the minor performance regression observed at 8 threads.

Task 2: Array Sorting (Parallel Bubble Sort / Odd-Even Transposition Sort)

1. Objective & Problem Statement

The goal of this task is to implement a parallel sorting algorithm using OpenMP. Since standard bubble sort is inherently sequential due to loop-carried dependencies, we implement Odd-Even Transposition Sort for array sizes N=50,000 and N=100,000, which parallelizes the sorting process by alternating between even and odd pairing phases.

2. Implementation Details

- **Compilation & Execution:** Compiled using g++ -fopenmp -O2 -Wall and execution time measured precisely via wall-clock timer omp_get_wtime(). Thread counts were managed dynamically using $env:OMP_NUM_THREADS.
- **Serial Version:** Baseline sorted using standard optimized C++ library sorting (std::sort).
- **Parallel Version:** Implemented using alternating even and odd pairing phases inside #pragma omp parallel for loops, allowing independent comparisons and swaps to occur concurrently across threads.
- **Verification:** Element-wise comparison between the parallel sorted array and the serial sorted baseline across all test runs, outputting SUCCESS.

3. Timing and Speedup Table

(Measured across array sizes N=50,000, 100,000 and thread counts P=1,2,4,8)

Size (N)	Threads	Tser (s)	Tpar (s)	Speedup	Status
50,000	1	0.0000	0.9640	0.00	SUCCESS
50,000	2	0.0000	1.0780	0.00	SUCCESS
50,000	4	0.0020	1.3780	0.00	SUCCESS

Size (N)	Threads	Tser (s)	Tpar (s)	Speedup	Status
50,000	8	0.0000	2.5120	0.00	SUCCESS
100,000	1	0.0000	3.7460	0.00	SUCCESS
100,000	2	0.0000	3.0710	0.00	SUCCESS
100,000	4	0.0000	2.7300	0.00	SUCCESS
100,000	8	0.0000	5.6570	0.00	SUCCESS

4. Observations

- **How is parallel bubble sort implemented, and what synchronization is required?**
  - **Answer:** Standard Bubble Sort cannot be parallelized directly because comparison and swap operations in step i depend strictly on the results of step i-1 (a loop-carried dependency). To overcome this, Odd-Even Transposition Sort divides each sorting iteration into two distinct phases: an even phase (comparing indices (0,1),(2,3),…) and an odd phase (comparing indices (1,2),(3,4),…). Synchronization Required: Because multiple threads evaluate and swap adjacent elements concurrently within the same phase, implicit barriers at the end of each #pragma omp parallel for construct are strictly required. These barriers ensure that all threads complete their comparisons and updates for the current phase before any thread proceeds to the next phase, preventing race conditions and ensuring data consistency across array elements.
- **Why does the parallel sorting algorithm show higher execution times compared to serial std::sort?**
  - **Answer:** The serial baseline relies on std::sort (IntroSort), which operates at an optimal O(N log N) time complexity. In contrast, Odd-Even Transposition Sort has an O(N^2) time complexity. Furthermore, the constant overhead of thread synchronization and barrier waits across thousands of alternating phases makes the parallel implementation computationally heavier than the highly tuned serial library sort for these specific array sizes.

Task 3: Histogram Calculation

1. Objective & Problem Statement

The goal of this task is to compute the frequency distribution of integer values ranging from 0 to 9 in a large array a[N] of size N=100,000,000, while preventing data races when multiple threads update shared frequency bins (hist[val]++) using OpenMP atomic operations.

2. Implementation Details

- **Compilation & Execution:** Compiled using g++ -fopenmp -O2 -Wall and execution time measured precisely via wall-clock timer omp_get_wtime(). Thread counts were managed dynamically using $env:OMP_NUM_THREADS.
- **Serial Version:** A sequential loop iterating through the array and incrementing corresponding bins in a local hist[10] array.

- **Parallel Version:** Parallelized using #pragma omp parallel for with #pragma omp atomic enforced on hist[val]++ to prevent race conditions during concurrent bin updates.
- **Verification:** Compares all 10 parallel frequency bins against the serial baseline across all test runs, outputting SUCCESS.

3. Timing and Speedup Table

(Measured across array size N=100,000,000 and thread counts P=1,2,4,8)

Size (N)	Threads	Tser (s)	Tpar (s)	Speedup	Status
10^8	1	0.0690	0.6380	0.11	SUCCESS
10^8	2	0.0550	1.3890	0.04	SUCCESS
10^8	4	0.0680	1.5260	0.04	SUCCESS
10^8	8	0.0420	1.7160	0.02	SUCCESS

4. Observations

- **Why does the parallel atomic histogram execution run significantly slower than the serial version?**
  - **Answer:** This performance degradation occurs due to heavy atomic contention (false sharing and serialization locks). Because there are only 10 unique histogram bins (0 to 9) shared globally across millions of loop iterations, multiple parallel threads constantly contend for the exact same few memory addresses to execute #pragma omp atomic increments. The hardware-level bus locking and serialization required to resolve these memory conflicts create massive synchronization overhead, making the parallel version considerably slower than a highly cache-optimized serial loop.

Task 4: Matrix Transpose

1. Objective & Problem Statement

The goal of this task is to transpose a 2D matrix of size N×N (N=10,000, total elements 10^8) by converting rows into columns (transposed[j * rows + i] = matrix[i * cols + j]) using OpenMP parallelization on the outer loop.

2. Implementation Details

- **Compilation & Execution:** Compiled using g++ -fopenmp -O2 -Wall and execution time measured precisely via wall-clock timer omp_get_wtime(). Thread counts were managed dynamically using $env:OMP_NUM_THREADS.
- **Serial Version:** Nested sequential loops over rows and columns using flat 1D raw pointers.
- **Parallel Version:** Parallelized using #pragma omp parallel for exclusively on the outer row loop.

- **Verification:** Element-wise comparison confirming that the parallel transposed matrix matches the serial baseline result identically, outputting SUCCESS.

3. Timing and Speedup Table

(Measured across matrix size 10,000×10,000 and thread counts P=1,2,4,8)

Size (N)	Threads	Tser (s)	Tpar (s)	Speedup	Status
10,000×10,000	1	0.9440	0.9360	1.01	SUCCESS
10,000×10,000	2	0.8910	0.4460	2.00	SUCCESS
10,000×10,000	4	0.8640	0.2370	3.65	SUCCESS
10,000×10,000	8	0.8570	0.1770	4.84	SUCCESS

4. Observations

- **How does outer-loop parallelization affect memory access patterns and scalability during matrix transposition?**
  - **Answer:** In a matrix transposition, reading the source matrix sequentially is cache-friendly (stride-1 access), whereas writing to the destination matrix involves strided memory accesses. Parallelizing the outer loop distributes independent rows across threads. As thread counts increase from 1 to 8, the workload is effectively partitioned, resulting in near-linear speedups (up to 4.84× on 8 threads) due to high parallel memory bandwidth utilization on large grids.

Task 5: 2D Convolution

1. Objective & Problem Statement

The goal of this task is to apply a 3×3 averaging filter kernel to an image represented as a matrix (tested on a large 4000×4000 image size) using valid filter positions with no padding, and parallelize the computation using OpenMP. 2D convolution is an essential image processing operation used for blurring, smoothing, edge detection, sharpening, and noise reduction where a small filter kernel slides over the larger image matrix to compute a new value for each position.

2. Implementation Details

- **Compilation & Execution:** Compiled using g++ -fopenmp -O2 -Wall and execution time measured precisely via wall-clock timer omp_get_wtime(). Thread counts were managed dynamically using $env:OMP_NUM_THREADS.
- **Serial Version:** Nested loops applying the pre-scaled 3×3 averaging filter weights (1.0/9.0) across valid image regions sequentially.
- **Parallel Version:** Parallelized using #pragma omp parallel for across the outer output rows to distribute the row-wise computation concurrently across multiple threads.
- **Verification:** Element-wise verification ensuring parallel convolution outputs match the serial baseline identically, outputting SUCCESS.

3. Timing and Speedup Table

(Measured across image size 4000×4000 and thread counts P=1,2,4,8)

Size (N)	Threads	Tser (s)	Tpar (s)	Speedup	Status
4000×4000	1	0.1130	0.0940	1.20	SUCCESS
4000×4000	2	0.1060	0.0570	1.86	SUCCESS
4000×4000	4	0.1150	0.0360	3.19	SUCCESS
4000×4000	8	0.1040	0.0260	4.00	SUCCESS

4. Observations

- **Why does 2D convolution exhibit high parallel efficiency and scaling?**
  - **Answer:** 2D convolution is an embarrassingly parallel operation when computing output positions over valid regions. Each thread calculates distinct output pixels using read-only access to the input image and private local accumulators (double sum), writing results to isolated memory locations in the output matrix. Because there are no shared write conflicts, data dependencies, or atomic locks required during inner loop computations, threads execute concurrently with near-zero synchronization overhead, yielding clean, scalable speedups (up to 4.00× on 8 threads).